

INDR 100

INTRODUCTION TO INDUSTRIAL ENGINEERING

Lecture #5

A Gallery Walk of Industrial Engineering Problems – Cont'd

Ceyda Oğuz
coguz@ku.edu.tr
Koç University

23

The Binary Switch

- We use Binary Variables (0 or 1).
- 1 = YES
- 0 = NO
- We turn the problem into a sequence of Yes/No decisions.

24

The NASA Payload (Knapsack)

- Scenario: Packing for Mars (Limit 10,000kg).
- Items: Oxygen, Water, Rover, Lab.
- The Trap: Intuition says 'Take the most valuable item first!'
- The Reality: That leaves gaps (Inefficiency).



This Photo by Unknown Author is licensed under CC BY-SA-NC

25

Knapsack problem

Motivation

- Many decision problems involve selecting the *best combination of items* under limited resources.
- Examples:
 - A company choosing projects within a budget.
 - A traveler packing valuable items in a suitcase with weight limit.
 - A delivery company accepting the most profitable shipments onto a truck.

26

Knapsack problem

Problem Definition

- Given a set of items I to choose from
- Each item $i \in I$ has :
 - **Value (profit)** p_i
 - **Weight (resource use)** w_i
- There is a total knapsack capacity: W
- Goal: Maximize the total value without exceeding the capacity

27

Knapsack problem

Mathematical Formulation

Define binary decision variable:

$$x_i = \begin{cases} 1, & \text{item } i \in I \text{ is selected} \\ 0, & \text{otherwise} \end{cases}$$

Objective:

$$\max \sum_{i \in I} p_i x_i$$

Subject to coverage constraints:

$$\sum_{i \in I} w_i x_i \leq W$$

28

Dynamic Programming (DP) algorithm to solve knapsack problem

- DP solves Knapsack by breaking it into subproblems.
- We build a table where rows represent items and columns represent capacities.
- Each cell stores the maximum value achievable for that subproblem.

29

Dynamic Programming (DP) algorithm to solve knapsack problem

Value Function

The value function represents:

- $V(i, w)$ = *maximum total value achievable using the first i items with left capacity w*
 i = number of items considered (from 1 to i)
 w = remaining capacity of the knapsack
- $V(i, w)$ answers: “What is the best value we can get if we only look at the first i items and have w units of capacity?”

30

Dynamic Programming (DP) algorithm to solve knapsack problem

Recursive relation

$$V(i, w) = \begin{cases} V(i-1, w), & \text{if } w_i > w \\ \max\{V(i-1, w), p_i + V(i-1, w - w_i)\}, & w_i \leq w \end{cases}$$

DP table:

- Rows = items,
- Columns = capacities.
- Build table bottom-up using recurrence.

31

Dynamic Programming (DP) algorithm to solve knapsack problem

Example:

- Items: [a, b, c], Weights: [1, 3, 4], Values: [15, 20, 30], Capacity: 4
- DP Table:

	0	1	2	3	4
Item-null	0	0	0	0	0
Item a	0	15	15	15	15
Item b	0	15	15	20	35
Item c	0	15	15	20	35

32

The ER Doctor (Scheduling)

- Scenario: 1 Doctor. 2 Patients.
- Patient A: Broken leg (60 mins).
- Patient B: Having fever (5 mins).
- Decision: First-Come-First-Served vs. Shortest-Job-First?

34

Minimizing Pain

- Sequencing matters.
- Changing the order reduces average waiting time.
- We use logic (Algorithms) to make this fair.

35

Job Scheduling

One printer (single machine) must process print a set of jobs J , before their due times. If a job finishes after its due time, it incurs a penalty.

Job	p_j	d_j	w_j
1	7	20	3
2	5	10	2
3	4	12	1
4	6	18	4
5	3	8	2

Goal: sequence the jobs on one printer to **minimize total weighted tardiness** ($\sum_{j \in J} w_j T_j$).

36

Job Scheduling

Decision variables

- $s_j \geq 0$: start time of job j
- $C_j \geq 0$: completion time of job j
- $T_j \geq 0$: tardiness of job j
- $z_{ij} \in \{0,1\} = 1$ if job i is scheduled **before** job j (for $i \neq j$)

$$\text{Let } M = \sum_{j \in J} p_j$$

Objective

$$\text{minimize } \sum_{j \in J} w_j T_j$$

No-overlap (single machine)

$$\begin{aligned} s_i &\geq s_j + p_j - M z_{ij} & \forall i \neq j \\ s_j &\geq s_i + p_i - M(1 - z_{ij}) & \forall j \neq i \\ C_j &= s_j + p_j & \forall j \\ T_j &\geq C_j - d_j & \forall j \end{aligned}$$

37

What have we seen so far?

- Models are the backbone of Industrial Engineering
- Mathematical programming models:
 - Decision variables: What we control
 - Parameters: What we cannot control → Data
 - Objective Function: What we want to make better, improve
 - Constraints: What limits us
 - Uncertainty: What we do not know